SIMATS ENGINEERING

Short Answer Test

COURSE NAME: Software Testing **COURSE CODE: CSA3712**

COURSE OUTCOME COVERED

CO1: *Apply software testing principles to enhance the effectiveness of software quality assurance and improve defect detection strategies.*

Assessment Tool Weightage: 30%

Sl. No	Questions	Bloom's Level
1	A startup team relies on random testing methods, leading to inconsistent results and missed defects. The project manager wants to improve testing effectiveness. As a test engineer, apply the concept of testing as an engineering activity to redesign the approach. Explain how structured test design improves quality and defect detection.	3
2	A banking system undergoes multiple failures due to lack of a defined testing process. The organization plans to standardize its testing activities. Illustrate how a well-defined testing process improves software quality in such real-time systems. Explain the benefits with relevant examples.	3
3	During development of an e-commerce platform, testing activities are performed inconsistently across phases. This leads to integration and system-level defects. Demonstrate how testing should be carried out as a structured process in SDLC. Explain how process discipline ensures better quality outcomes.	3
4	A login system shows mismatch between expected and actual outputs during validation. The development team is unsure whether the issue is an error, defect, or failure. Apply basic testing definitions to classify the issue. Explain the relationship between these terms with suitable examples.	3
5	A team struggles to detect defects early, leading to increased cost during later stages. The manager wants to improve defect detection efficiency. Apply a software testing principle to address this issue. Explain how the principle improves testing effectiveness with an example.	3
6	During requirement analysis of a healthcare system, several ambiguities go unnoticed. Later, these lead to defects in implementation. Show how a tester can contribute during the requirement phase to improve quality. Explain the importance of early testing involvement.	3
7	A payroll system produces incorrect outputs due to unclear and ambiguous requirements. The development team needs to identify the origin of defects. Analyze how such defects originate in the software lifecycle. Explain how requirement clarity can prevent such issues.	3

8	A system crashes when boundary input values are entered by users. The team suspects improper testing coverage. Classify this defect and justify your answer. Explain how boundary value analysis could have prevented this issue.	3
9	A company maintains a defect repository but rarely uses it for improving testing practices. This results in repeated defects across releases. Demonstrate how a defect repository can support better test case design. Explain its role in improving software quality.	3
10	An ATM system fails during withdrawal due to incorrect handling of transaction logic. The issue was not detected during testing. Apply your testing knowledge to identify and describe this as a functional defect. Explain how it could have been prevented.	3
11	A large application shows that most defects are concentrated in a few modules. The testing team wants to optimize their efforts. Use the concept of defect clustering to identify testing priorities. Explain how this improves testing efficiency.	3
12	A web application experiences slow performance under heavy user load. The issue needs to be categorized properly. Apply defect classification techniques to identify this type of defect. Explain how classification helps in better resolution.	3
13	A team is preparing for regression testing after multiple releases. They have access to historical defect data. Show how this data can be reused effectively for regression testing. Explain how it improves test coverage and efficiency.	3
14	During integration testing, multiple defects are found due to poor communication between developers and testers. The organization wants to improve collaboration. Identify the role of a tester in minimizing such defects. Explain how early involvement helps.	3
15	A software product repeatedly faces similar defects in every release cycle. The team needs a long-term solution. Apply defect prevention techniques to address recurring issues. Explain how prevention improves overall software quality.	3
16	A mobile application frequently crashes due to inconsistent testing approaches. The testing team wants to standardize practices. Apply testing principles to redesign the testing strategy. Explain how structured approaches improve reliability.	3
17	A company confuses debugging with testing, leading to delays in defect resolution. The team needs clarity on both concepts. Differentiate testing and debugging using a real-time scenario. Explain their roles in improving software quality.	3
18	An online shopping system behaves differently across browsers. This leads to customer dissatisfaction. Apply testing principles to design a cross-browser testing strategy. Explain how such issues can be effectively handled.	3

19	A logistics application produces inconsistent outputs due to improper validation. The issue is not detected early. Apply testing concepts to identify gaps in validation. Explain how proper testing improves system reliability.	3
20	A team skips structured test case design and directly executes random tests. This leads to poor defect detection. Apply testing as an engineering activity to improve the process. Explain the importance of systematic test design.	3
21	A healthcare system shows data inconsistency between modules due to integration issues. The testing team needs to analyze the problem. Apply testing principles to identify root causes. Explain how proper testing ensures data consistency.	3
22	A project experiences delays because testers are not involved early in development. The organization wants to improve collaboration. Explain the role of testers across SDLC phases. Show how early involvement improves quality.	3
23	A system fails during peak usage due to untested scenarios. The testing team missed critical cases. Apply testing principles to identify gaps. Explain how better test coverage prevents such failures.	3
24	A software team lacks proper defect documentation, making tracking difficult. The organization wants to improve defect management. Apply defect repository concepts to solve this issue. Explain how documentation enhances tracking and analysis.	3
25	A banking system produces repeated transaction failures due to improper testing practices. The team needs to improve quality. Apply software testing principles to redesign the testing approach. Explain how structured testing enhances reliability.	3
26	A system shows failures due to missing edge case testing. The testers focused only on normal inputs. Apply testing principles to include edge cases. Explain how boundary testing improves defect detection.	3
27	A development team struggles to maintain consistency in testing across modules. This leads to uneven quality. Apply process-based testing approaches to address this issue. Explain how standardization improves outcomes.	3
28	A web application fails due to improper handling of unexpected inputs. The testing team missed validation scenarios. Apply testing concepts to improve input validation. Explain how proper testing ensures robustness.	3
29	A software company faces frequent defect leakage into production. The management wants to reduce such issues. Apply testing principles to analyze the root cause. Explain how improved testing practices prevent leakage.	3
30	A project team is unable to identify defects early due to poor testing strategies. This increases overall cost and effort. Apply testing principles to redesign the strategy. Explain how early defect detection improves efficiency.	3

1. Testing as an Engineering Activity

Testing as an engineering activity is a systematic process used to identify defects and improve software quality.

- Prepare test cases based on requirements.
- Define expected results and test valid, invalid, and boundary inputs.
- Use tools such as Selenium and Jira to automate tests and track defects.

Example: In a banking application, test valid and invalid money transfers.

Conclusion: Structured testing improves defect detection, coverage, and software quality.

2. Well-Defined Testing Process

A well-defined testing process ensures that testing activities follow a planned procedure.

- Analyze requirements and prepare a test plan.
- Design, execute, and evaluate test cases.
- Report defects, retest fixes, and perform regression testing.

Example: Test ATM withdrawals for incorrect PINs and insufficient balance.

Conclusion: A standard process reduces failures and improves reliability.

3. Testing Process in SDLC

Testing should be performed systematically throughout the Software Development Life Cycle (SDLC).

- Requirement analysis identifies missing requirements.
- Test design prepares test cases.
- Integration and system testing check combined components and complete functionality.
- Regression testing verifies existing features after changes.

Example: Test product selection, cart, and payment integration in an e-commerce application.

Conclusion: Process discipline detects defects early and improves software quality.

4. Error, Defect, and Failure

- Error: A human mistake during development.
- Defect: A flaw in the software caused by an error.
- Failure: Software behaves incorrectly during execution.

Example: A developer writes incorrect login logic (error), creating a code defect. When a valid user cannot log in, a failure occurs.

Conclusion: Understanding these terms improves defect reporting and problem resolution.

5. Early Testing Principle

Early testing means starting testing activities as soon as possible in the development lifecycle.

- Review requirements and design early.
- Identify defects before coding or deployment.
- Correcting defects early reduces repair costs.

Example: Identifying an incorrect payment requirement during review prevents expensive corrections later.

Conclusion: Early testing saves time, reduces costs, and improves software quality.

6. Tester's Role in Requirement Analysis

Testers should participate in requirement analysis to identify ambiguities and missing requirements.

- Review requirements for completeness.
- Ask questions about unclear conditions.
- Prepare acceptance criteria and initial test scenarios.

Example: In a healthcare system, clarify how duplicate patient records should be handled.

Conclusion: Early tester involvement prevents misunderstandings and reduces defects.

7. Defects Caused by Unclear Requirements

Defects may originate from ambiguous requirements, design mistakes, coding errors, or inadequate testing.

- Unclear requirements cause incorrect implementation.
- Developers may interpret the same requirement differently.
- Requirement reviews help prevent these issues.

Example: An unclear payroll tax rule results in incorrect salary calculations.

Conclusion: Clear, complete, and testable requirements reduce defects and rework.

8. Boundary Value Analysis

A system crash caused by boundary inputs may indicate a validation or boundary-handling defect.

- Test minimum and maximum permitted values.
- Test values immediately below and above the limits.

- Record unexpected results and fix the defect.

Example: If ages from 18 to 60 are accepted, test 17, 18, 19, 59, 60, and 61.

Conclusion: Boundary Value Analysis detects edge-case defects and improves test coverage.

9. Defect Repository

A defect repository stores historical defect information and supports better test case design.

- Record defect descriptions, severity, root cause, and resolution.
- Analyze repeated defect patterns.
- Create regression tests based on previous defects.

Example: Previous login failures can be used to design additional login validation tests.

Conclusion: A defect repository prevents recurring defects and improves testing efficiency.

10. Functional Defect in an ATM

A functional defect occurs when a feature does not behave according to its requirements.

- Incorrect transaction logic may cause withdrawal failures.
- Test valid withdrawals, insufficient balances, and withdrawal limits.
- Report defects and verify corrections.

Example: An ATM deducts money but does not dispense cash.

Conclusion: Functional testing and transaction validation help prevent banking errors.

11. Defect Clustering

Defect clustering means that most defects are concentrated in a few software modules.

- Identify modules with a history of defects.
- Prioritize high-risk and frequently used modules.
- Allocate more testing effort to these areas.

Example: If most banking defects occur in fund transfers, test transaction calculations and validation first.

Conclusion: Defect clustering improves testing efficiency and helps detect critical defects early.

12. Performance Defect

A performance defect occurs when software responds slowly or cannot handle the expected workload.

- Perform load testing and stress testing.

- Monitor response time, memory, and server usage.
- Identify and fix performance bottlenecks.

Example: An online shopping website becomes slow when thousands of users access it simultaneously.

Conclusion: Performance testing improves speed, stability, and user satisfaction.

13. Historical Defect Data in Regression Testing

Historical defect data helps testers identify areas that require repeated testing after software changes.

- Review previous defect reports.
- Select test cases for frequently affected modules.
- Automate repeated regression tests where suitable.
- Verify that old defects have not returned.

Example: Previous payment failures are included in regression tests after updating the payment module.

Conclusion: Historical data improves test coverage and saves testing time.

14. Tester Collaboration During Integration

Testers help developers identify defects when different software components interact.

- Participate in requirement and design reviews.
- Communicate expected behavior clearly.
- Report integration defects with reproduction steps.
- Verify fixes through retesting.

Example: A tester identifies that the payment module does not receive the correct order details from the cart module.

Conclusion: Early involvement and collaboration reduce integration defects and improve quality.

15. Defect Prevention

Defect prevention focuses on avoiding defects before they occur.

- Analyze the root causes of repeated defects.
- Improve coding standards and requirement reviews.
- Conduct code reviews and developer training.
- Maintain regression tests for previous defects.

Example: Repeated calculation errors can be prevented through formula reviews and automated tests.

Conclusion: Defect prevention reduces rework, lowers costs, and improves long-term software quality.

16. Standardized Testing for Mobile Applications

Testing principles help establish a consistent approach to mobile application testing.

- Prepare a test plan and structured test cases.
- Test different devices, screen sizes, and network conditions.
- Perform functional, compatibility, and performance testing.
- Track defects and conduct regression testing.

Example: Test a mobile banking app during network disconnection and reconnection.

Conclusion: Structured testing reduces crashes and improves mobile application reliability.

17. Testing vs Debugging

Testing identifies defects, while debugging finds their root causes and fixes them.

- Testing compares actual and expected results.
- Debugging investigates incorrect code or logic.
- After debugging, testers retest the software.

Example: A tester reports that login rejects a valid password. The developer finds and fixes the incorrect validation condition.

Conclusion: Testing and debugging work together to improve software quality.

18. Cross-Browser Testing

Cross-browser testing ensures that a web application works correctly in different browsers.

- Test in Chrome, Firefox, and Edge.
- Check layouts, forms, buttons, and payment functions.
- Use Selenium or BrowserStack for browser compatibility testing.
- Fix browser-specific defects and retest.

Example: A payment button works in Chrome but fails in Firefox.

Conclusion: Cross-browser testing improves compatibility and user experience.

19. Validation in a Logistics Application

Input validation ensures that the application accepts valid data and handles invalid data correctly.

- Identify missing validation rules.

- Test valid, invalid, empty, and unexpected inputs.
- Compare actual outputs with expected results.
- Report and fix defects.

Example: A logistics application accepts an invalid postal code and assigns a package to the wrong delivery area.

Conclusion: Proper validation prevents incorrect outputs and improves system reliability.

20. Systematic Test Case Design

Testing as an engineering activity replaces random testing with planned and documented procedures.

- Understand requirements.
- Prepare test cases with expected results.
- Include valid, invalid, and boundary inputs.
- Execute tests, report defects, and verify fixes.

Example: Test banking withdrawals using valid amounts, insufficient balances, and withdrawal limits.

Conclusion: Systematic test design improves defect detection, coverage, and testing efficiency.

21. Data Inconsistency Between Healthcare Modules

Data inconsistency occurs when different modules store or display conflicting information.

- Check data flow between modules.
- Perform integration testing.
- Validate database updates and synchronization.
- Track defects and retest corrections.

Example: A patient's updated address appears in registration but not in billing.

Conclusion: Integration testing maintains data consistency and improves healthcare system reliability.

22. Tester's Role Across SDLC

Testers contribute to quality at every stage of the SDLC.

- Requirements: Review requirements and identify ambiguity.
- Design: Review workflows and prepare test scenarios.
- Development: Support unit and integration testing.
- Testing: Execute test cases and report defects.

- Maintenance: Perform regression testing.

Example: A tester identifies missing payment validation during design review**.**

Conclusion: Early and continuous tester involvement reduces rework and improves quality.

23. Testing Under Peak Usage

System failures during peak usage indicate that performance scenarios may not have been tested adequately.

- Identify expected maximum user load.
- Perform load and stress testing.
- Monitor response time and resource usage.
- Include peak-hour scenarios in test plans.

Example: An online examination portal crashes when hundreds of students log in simultaneously**.**

Conclusion: Better test coverage identifies performance problems before deployment and improves reliability.

24. Defect Documentation and Repository

A defect repository stores and organizes defect information for tracking and analysis.

- Record defect ID, description, severity, and priority.
- Include reproduction steps and expected versus actual results.
- Update the status and resolution.
- Analyze historical defects to prevent recurrence.

Example: Jira can track a delivery-status defect from reporting to resolution.

Conclusion: Proper documentation improves communication, tracking, and defect management.

25. Testing a Banking Transaction System

Structured testing helps identify transaction failures and ensure accurate processing.

- Analyze requirements and prepare test cases.
- Test valid and invalid transactions.
- Perform integration and regression testing.
- Check duplicate transactions and network failures.
- Record defects and verify corrections.

Example: A transfer deducts money twice because the system processes the same request twice.

Conclusion: Systematic testing reduces transaction failures and improves banking reliability.

26. Edge Case Testing

Edge cases are unusual or extreme input conditions that may cause software failures.

- Identify minimum and maximum values.
- Test values just below and above boundaries.
- Include empty, invalid, and unexpected inputs.
- Perform regression testing after fixes.

Example: For an accepted amount between ₹100 and ₹10,000, test ₹99, ₹100, ₹101, ₹9,999, ₹10,000, and ₹10,001.

Conclusion: Boundary testing detects edge-case defects and improves software robustness.

27. Process-Based Testing

Process-based testing follows standardized procedures across all software modules.

- Use a common test plan and test case format.
- Follow consistent execution procedures.
- Maintain standard defect-reporting practices.
- Review results and perform regression testing.

Example: Login, payment, and order modules use the same test documentation and reporting process.

Conclusion: Standardization improves consistency, reduces confusion, and ensures better testing outcomes.

28. Input Validation Testing

Input validation testing checks whether software handles valid and unexpected inputs correctly.

- Test valid and invalid data.
- Check empty fields, incorrect formats, and boundary values.
- Validate data types and permitted ranges.
- Verify that invalid inputs produce appropriate error messages.

Example: A registration form should reject an invalid email address and prevent submission until it is corrected.

Conclusion: Proper input validation prevents incorrect processing and improves software robustness and security.

29. Defect Leakage into Production

Defect leakage occurs when defects are missed during testing and are discovered after software deployment.

Causes:

- Incomplete test coverage
- Insufficient regression testing
- Missing edge cases
- Poor requirement understanding

Prevention:

- Improve test planning and coverage.
- Perform integration and system testing.
- Include risk-based and regression testing.
- Analyze production defects and update test cases.

Example: A payment defect is discovered by customers because duplicate transactions were not tested before release.

Conclusion: Improved testing practices reduce production defects, maintenance costs, and customer dissatisfaction.

30. Early Defect Detection

Early defect detection identifies problems during the initial stages of software development.

- Review requirements and design documents.
- Involve testers from the beginning.
- Prepare test cases early.
- Use code reviews and automated tests.
- Track defects and perform regression testing.

Example: Identifying an incorrect salary calculation requirement during review prevents coding errors and expensive corrections later.

Conclusion: Early defect detection reduces development costs, saves time, and improves software quality.

Code of Conduct:

*I **Sadhana K / 192421137** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: *sadhana K*

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	**30**	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	**25**	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	**20**	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	**15**	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	**10**	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand